Pandas Cheat Sheet (Module 6)

`import pandas as pd`. All examples use ShopKart files. Numbers in comments are real results.

Load, save, inspect

```
orders = pd.read_csv("orders_raw.csv")                    # (5055, 12)
df = pd.read_csv("shopkart_clean.csv", parse_dates=["order_date"])   # keep dates as dates
df.to_csv("shopkart_clean.csv", index=False)              # no extra index column

df.head(), df.tail(3)        # first / last rows
df.shape                     # (rows, columns)
df.columns                   # column names
df.info()                    # types + non-null counts
df.describe()                # number summary (max quantity 99 = suspicious)
df.isnull().sum()            # blanks per column
df.duplicated().sum()        # exact duplicate rows (55)
df["city"].nunique()         # distinct values (27 raw spellings)
df["city"].value_counts()    # count per value; normalize=True for shares; dropna=False shows NaN
```

Select

```
df["city"]                           # one column (Series)
df[["order_id", "city", "revenue"]] # several columns (DataFrame) - double brackets
df.iloc[0]                           # row by position
df.loc[0:2, ["order_id", "quantity"]]   # rows by label (end included), chosen columns
```

Filter and sort

```
df[df["payment_mode"] == "COD"]
df[(df["delivery_days"] >= 8) & (df["payment_mode"] == "COD")]   # & AND, | OR, ~ NOT; brackets!
df[df["city"].isin(["Pune", "Delhi"])]
df[df["rating"].isnull()]            # rows with missing rating
df.sort_values("revenue", ascending=False)
df.sort_values(["city", "rating"], ascending=[True, False])
df.nlargest(5, "quantity")
```

Excel	SQL	pandas
Filter	WHERE	df[condition]
Sort	ORDER BY	sort_values
COUNTIF	COUNT(*) ... GROUP BY	value_counts()

Excel	SQL	pandas
SUMIFS / pivot	`GROUP BY`	`groupby` / `pivot_table`
XLOOKUP	`LEFT JOIN`	`merge(how="left")` / `map`

Clean

```
df = df.drop_duplicates()                                    # always
assign back!
df["city"] = df["city"].str.strip().str.title()              # text
tools via .str
df["city"] = df["city"].replace({"Bombay": "Mumbai", "Madras":
"Chennai"})   # whole values
df["unit_price"] = (df["unit_price"].astype(str)
                    .str.replace("₹", "", regex=False)
                    .str.replace(",", "", regex=False)
                    .astype(float).astype(int))              # "₹1,299"
-> 1299
s_ = df["order_date"].astype(str)                            # two date
styles in one column:
df["order_date"] = pd.to_datetime(s_, format="%Y-%m-%d",
errors="coerce").fillna(
                   pd.to_datetime(s_, format="%d-%m-%Y",
errors="coerce"))   # exact format each
df["order_date"].isnull().sum()                              # check: 0
dates failed
# AVOID format="mixed", dayfirst=True: it silently turns 2025-01-02
into 1 Feb
df["payment_mode"] = df["payment_mode"].fillna("Unknown")   # fill
when you know the meaning
df["discount"] = df["discount"].fillna(0.0)
df.loc[df["quantity"] == 99, "quantity"] = 1                 # change
matching rows only
df = df.rename(columns={"old": "new"})
df = df.drop(columns=["col"])
```

Rule: fill a blank only when you know what it means (discount blank = 0). Keep genuine missing values (rating blank = not rated). Before fixing an outlier, look at what it is.

New columns

```
df["revenue"] = (df["quantity"] * df["unit_price"] * (1 -
df["discount"])).round(2)
df["order_month"] = df["order_date"].dt.month          #
also .dt.month_name(), .dt.day_name(), .dt.year
df["is_returned"] = (df["returned"] == "Y").astype(int) # mean of 0/1
= rate
df["order_size"] = df["revenue"].apply(order_size)     # run your own
function on every value
df["age_group"] = pd.cut(df["age"], bins=[17, 25, 35, 45, 60],
                         labels=["18-25", "26-35", "36-45", "46-60"])
# right edge included
```

```
cost = df["product_id"].map(products.set_index("product_id")
["cost_price"])  # lookup
```

Summarise

```
df.groupby("city")["revenue"].sum().sort_values(ascending=False)
df.groupby("city").agg(orders=("order_id", "count"),
                       revenue=("revenue", "sum"),
                       avg_rating=("rating", "mean"),
                       return_rate=("is_returned", "mean"))
df.groupby(["city", "payment_mode"])["is_returned"].mean()
pd.pivot_table(df, index="city", columns="payment_mode",
values="revenue",
               aggfunc="sum", margins=True)
df["discount"].corr(df["quantity"])     # +0.27
df["customer_id"].nunique()             # 758 customers ordered
```

Statistics skip NaN automatically: `mean`, `sum`, `corr`, `count`.

Merge

```
products["product_id"].duplicated().sum()     # check: must be 0
df = orders.merge(products[["product_id", "category", "cost_price"]],
on="product_id", how="left")
df = df.merge(customers[["customer_id", "membership"]],
on="customer_id", how="left")
assert len(df) == 5000                        # row count must not
change
customers.merge(orders[["customer_id"]].drop_duplicates(),
on="customer_id",
                how="left", indicator=True)["_merge"].value_counts()
# left_only = 42 never ordered
```

how=: `"left"` (keep all left rows), `"inner"` (matches only), `"right"`, `"outer"`. Columns ending `_x` / `_y` mean you merged twice.

Quick chart (Module 8 goes deeper)

```
df.groupby("order_month")["revenue"].sum().plot(kind="bar",
title="Revenue by month");
```

Common errors

Error	Cause	Fix
`KeyError: 'Revenue'`	Column name typo / capitals	Check `df.columns`
`KeyError: ('a', 'b')`	`df["a", "b"]`	Use `df[["a", "b"]]`
`The truth value of a Series is ambiguous`	`and`/`or` with columns	Use `&` / `|`
`Can only use .dt accessor with datetimelike values`	Dates still text	`pd.to_datetime` or `parse_dates=`
Table unchanged after a method	Result not stored	`df =`

Error	Cause	Fix
		`df.drop_duplicates()`
`FileNotFoundError`	File not in Colab Files panel	Upload again
No error, but months look odd	`dayfirst=True` flipped ISO dates	Parse each exact `format=` separately

EDA finding template

Answer → number → comparison → so what → caution. "Pune returns 32.4% of orders, 2.4 times the 13.7% average, across every category. Check the delivery partner first."